

SYNOPSIS

1. Title of the Project

Retrivo — A Web-Based Lost and Found Management System for Educational Institutions.

2. Statement about the Problem

Educational institutions generally rely on manual lost-and-found processes such as security offices, notice boards, and informal messaging groups. These approaches are unorganized, difficult to search, and provide no reliable verification of ownership before an item changes hands. As a result, many lost items remain unclaimed, records are not maintained properly, and administrators have no way to audit outcomes. The absence of a centralized digital platform connecting the reporter, the finder/claimant, and the administrator creates delays, duplicate reports, and a higher risk of items being handed to the wrong person.

3. Why the Particular Topic is Chosen — Present State of the Art

This topic addresses a recurring campus problem while demonstrating a complete software engineering lifecycle — from requirement analysis to a working, deployed web application. Existing solutions such as physical counters and social media groups lack centralized records, search functionality, and administrative oversight. Commercial lost-and-found platforms are built for high-volume transit systems and airports, making them unsuitable and disproportionately complex for a single institution. Retrivo fills this gap with a centralized, role-based web application tailored for a campus setting, where administrators maintain the quality of listings and monitor how the platform is being used.

4. Objective and Scope of the Project

4.1 Objectives

Provide a single centralized platform to report lost or found items, replacing informal, scattered channels. Enable users to claim found items and respond when a lost item has been located. Give every user a personal dashboard of their reports, claims, and outcome statistics. Maintain a complete status history for every item — Open, Pending, Resolved.

Give administrators a system-wide statistics view and the ability to remove fake, duplicate, or vague item listings so that the data on the platform stays reliable.

4.2 Scope

The scope covers three roles — general user (reporter/claimant), administrator, and the underlying data layer — and the functions of registration/login, creating and browsing reports, submitting claims/responses, per-user dashboards, and an administrator view for platform statistics and listing moderation. Real-time push notifications, payment/courier-based return services, and external identity integration are excluded from the current build and treated as future extensions.

4.3 Primary Actors

- User (Reporter) — reports an item as Lost or Found.
- Claimant / Finder — responds to another user's report, either claiming a Found item or reporting that a Lost item has been found; a per-interaction role, not a separate account type.
- Administrator — monitors platform-wide usage statistics and removes fake, duplicate, or vague item listings; does not approve or reject individual claims.

4.4 Major Functional Modules

- Authentication Module — registration, login, JWT-based sessions, BCrypt password hashing, role-based access (user/admin).
- Dashboard & Analytics Module — per-user summary of reports, claims, and status statistics.
- Lost/Found Reports Module — creating, browsing, and searching item reports.
- Claims Management Module — submitting and tracking claims/responses between reporter and finder/claimant.
- Admin Panel Module — a statistics view of platform-wide activity (reports, claims, categories) and a moderation queue for deleting fake, duplicate, or vague item listings.

- Category Management Module — maintains item categories used to classify and filter reports.

5. Analysis, Design, Development & Testing Methodology

5.1 SDLC Model to be used

The project follows the Agile Software Development Model, where functional modules such as authentication, lost and found management, claims, administrator management, and dashboards are developed and deployed through iterative development cycles.

5.2 Justification for the Selection of Model

Agile was selected because its iterative cycles allow continuous testing, early bug detection, and incremental deployment. Its flexibility accommodated mid-development changes — such as migrating the database engine and moving from a local setup to a cloud-hosted deployment — without requiring the entire system to be re-specified, which is essential for a student-built, continuously evolving application like Retrivo.

5.3 Analysis

Requirements were derived by walking through the real sequence of events when an item is lost, mapping real-world events to core entities (Users, Items, Claims, Categories) and their relationships. The model uses one unified Claims table, distinguishing between ownership claims and finder responses through conditional interface labeling, with the reporter and claimant resolving a claim directly between themselves.

5.4 Design

The system uses a layered, three-tier architecture: an Angular single-page application communicates over HTTPS with an ASP.NET Core Web API, which persists data to a PostgreSQL database via Entity Framework Core (Npgsql provider). The backend follows RESTful API principles with JWT authentication and BCrypt password hashing. Four interconnected tables form the core data model regulating item status (Open, Pending, Resolved), driven by the reporter's and claimant's own actions, while a separate set of queries powers the administrator's statistics view and listing-moderation queue. Version control is maintained through Git, with the frontend and backend tracked within a single repository structure.

5.5 Development

Development began with database design and authentication, followed by item management, claims workflows, and reporting features. Modules were iteratively integrated with the existing system and validated using Postman and browser-based testing before advancing to the next feature set. The completed application was deployed to a cloud hosting platform so it could be accessed and tested over the internet rather than remaining a local prototype.

5.6 Testing Methodology

Testing spans backend unit-level checks, API integration testing via Postman across valid and invalid inputs, and manual end-to-end user-acceptance testing for complete workflows. Authentication, CRUD operations, API responses, form validations, and the admin statistics/moderation views are rigorously tested to ensure application reliability.

6. Hardware & Software to be Used

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
RAM	8 GB minimum (16 GB recommended)
Storage	10 GB free disk space (SSD recommended)

Table No-1: Hardware Specification (Minimum)

Layer	Technology
OS	Windows 10/11 (development)
Front End	Angular, TypeScript, Tailwind CSS
Back End	ASP.NET Core Web API (.NET)
Database / ORM	PostgreSQL, Entity Framework Core (Npgsql provider)
Authentication	JWT, BCrypt password hashing
IDE / Tools	Visual Studio / VS Code, DBeaver
API Testing	Postman, Swagger UI
Version Control	Git, GitHub
Hosting	Cloud-hosted deployment (Render)

Table No-2: Software Specification

7. Testing Technologies to be Used

The testing approach utilizes Postman for request/response testing of every API endpoint, including authentication and status-cascade logic. Swagger UI is used for interactive endpoint verification during backend development. Browser DevTools are utilized for inspecting front-end API calls, JWT handling, and client-side errors. DBeaver is used for

verifying data integrity directly at the PostgreSQL database level after test actions. Finally, manual end-to-end walkthroughs across seeded test accounts are conducted to validate full workflows such as reporting, claiming, dashboard updates, and admin moderation.

8. Contribution / Value Addition

Retrivo's principal contribution is bringing together the two real-world directions of the problem — claiming a found item, and reporting that a lost item was found — into one consistent data model with role-appropriate interface labeling, instead of treating them as separate flows. The personal dashboard adds visibility — a persistent record of what a user reported and claimed, with derived statistics unavailable in a manual process. The administrator's statistics and moderation view keeps the platform's data trustworthy by surfacing platform-wide usage at a glance and letting fake, duplicate, or vague listings be removed before they clutter genuine reports. Together these turn an ad hoc campus process into a structured, searchable one, providing a scalable foundation for future enhancements while demonstrating industry-standard full-stack development practices using Angular, ASP.NET Core, PostgreSQL, EF Core, and JWT.

9. Limitations / Constraints of the Project

The project has several limitations. There are no real-time notifications, meaning a user must revisit their dashboard to see status changes. There is no automated identity verification for a claim — resolution relies on the claimant's written description exchanged with the reporter, not biometric or document proof, and the platform does not adjudicate individual claims. It uses a single-institution deployment model, not built for multi-tenant use. There is no in-app messaging, so handover coordination relies on manually shared contact details. Resolved-state confirmation depends on reporter action; an item can stay Pending indefinitely if they never confirm the return.

10. Conclusion, Future Scope for Modification

10.1 Conclusion

Retrivo replaces an unstructured, unauditable manual process with a centralized web application giving every reported item a clear lifecycle — Open, Pending, and eventually

Resolved — with administrators overseeing platform-wide statistics and the genuineness of listings. The project demonstrates, in one application, the practical use of relational database design, secure authentication, REST API development, and a role-aware front end, applied to a real, recurring campus problem.

10.2 Future Scope

Email/in-app push notifications on claim submission or resolution. In-app messaging between claimant and reporter/finder. AI-assisted recommendation of similar lost and found reports based on category, description, and uploaded images. An AI chatbot to help users report items, answer common questions, and guide them through the claiming process.

11. References and Bibliography

- [1] Microsoft Learn. "ASP.NET Core Documentation." <https://learn.microsoft.com/aspnet/core/>
- [2] Angular Team. "Angular Official Documentation." <https://angular.dev/>
- [3] Microsoft Learn. "Entity Framework Core Documentation." <https://learn.microsoft.com/ef/core/>
- [4] PostgreSQL Global Development Group. "PostgreSQL Documentation." <https://www.postgresql.org/docs/>
- [5] JWT.io. "Introduction to JSON Web Tokens." <https://jwt.io/introduction>
- [6] Postman Inc. "Postman Learning Center." <https://learning.postman.com/>
- [7] Render. "Render Documentation." <https://render.com/docs>
- [8] Sommerville, I. Software Engineering (10th ed.). Pearson Education.
- [9] Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems (7th ed.). Pearson Education.